

Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start!
- Let's Explore!
- Let's Practice!
- Extension Activity
- Target Words
- Learning Points



Let's Start!

Are There Any Other Ways to Translate Source Code?

```

IF var.tipoconcepto = FALSE THEN
  IF Mid$(Upper$(var.concepto_codigo[numero]), 1, 1) = "0" THEN
    '1 mano de obra
    ME.supergridviewsPartida[fila - 1, 1].picture = Picture["casco28x28.png"]
    ME.supergridviewsPartida[fila - 1, 1].text = 1
  ELSE
    IF Mid$(Upper$(var.concepto_codigo[numero]), 1, 1) = "M" THEN
      '2 maquinaria
      ME.supergridviewsPartida[fila - 1, 1].picture = Picture["icono_retroexcavadora28x28.png"]
      ME.supergridviewsPartida[fila - 1, 1].text = 2
    ELSE
      IF Mid$(Upper$(var.concepto_codigo[numero]), 1, 1) = "P" THEN
        '3 materiales
        ME.supergridviewsPartida[fila - 1, 1].picture = Picture["iconomateriales28x28.png"]
        ME.supergridviewsPartida[fila - 1, 1].text = 3
      ELSE
        '0 otros
        ME.supergridviewsPartida[fila - 1, 1].picture = Picture["descomp.png"]
        ME.supergridviewsPartida[fila - 1, 1].text = 0
      ENDIF
    ENDIF
  ENDIF
ENDIF
IF var.tipoconcepto = TRUE THEN
  IF var.concepto_tipo[numero] = 1 THEN
    '1 mano de obra
    ME.supergridviewsPartida[fila - 1, 1].picture = Picture["casco28x28.png"]
    ME.supergridviewsPartida[fila - 1, 1].text = 1
  ELSE

```

Learning Outcome:

Describe common simple incidents at work in a simple spoken or written situation with a supportive interlocutor.

Represent the solution outline to the problem presented by the customer through making case of use diagrams, facing the problem to the customer following rules and protocols of the organization.

Ask your partner:

- How do you transform source code into another computer language?
- What types of compilers do you know about?

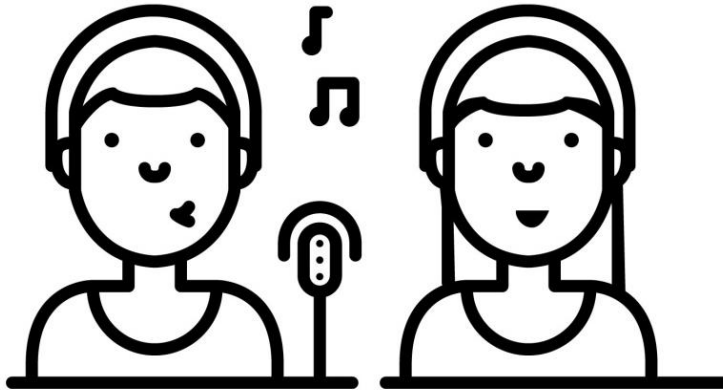
Can Do Statements:

- I can say what an interpreter is.
- I can list variations of different types of interpreters.
- I can talk about applications for interpreters.



Let's Explore!

Professor Miller is having an interview about interpreters in a radio college station.



Instructions: Read the text below.

- Radio host:** Hello, everyone, and welcome once more to our radio program, "Machine language". Today, our guest is professor Miller. He is going to solve all our doubts about interpreters. Good morning, professor, and thanks for accepting our invitation.
- Professor Miller:** Thanks for having me. It's a pleasure to be here with the college audience.
- Radio host:** Ok, professor, could you tell us... what an interpreter is?
- Professor Miller:** An interpreter is a program that can run encoded data without passing through a compilation process before.
- Radio host:** How is that possible? Is there any translation process?
- Professor Miller:** Yes, some interpreters are able to translate a source code language into an intermediate language and they can execute instructions immediately like LISP and Darmouth Basic.
- Radio host:** I see, is it always like that? Or are there any other features?
- Professor Miller:** Interpreters don't always translate. In some cases, they parse the code and they can set its behavior instantly or sometimes an interpreter can also run precompiled codes.
- Radio host:** Ok. Do you know some examples of those interpreters?
- Professor miller:** Well, there is a bunch of interpreters I know about. We have LISP and Darmouth Basic among those interpreters that do parsing, and Phyton and Ruby which can work for precompiled data. There are some programs like Smalltalk that can create intermediate representations and execute precompiled information at the same time.
- Radio host:** Are there any specific applications referring to interpreters?
- Professor Miller:** Yes, there are mainly four applications that concern them. First, interpreters can run command languages or glue languages. Second, they make easier to apply self-modifying codes. Third, they allow virtualization and lastly they apply relevant security standards with sandboxing technology.
- Radio host:** I see exactly what you mean...



Let's Practice!

There are different types of interpreters.

*Instructions: Match the questions in column A with the answers in column B.
Read the sentences aloud. Check your pronunciation with your instructor.*

COLUMN A

1. Is there any translation process?
2. Are there any other features?
3. Is there a bunch of interpreters that you know?

COLUMN B

- a. Interpreters don't always translate. In some cases, they parse the code.
- b. We have LISP and Darmouth Basic among those interpreters that do parsing and Phyton and Ruby can work for precompiled data.
- c. Yes, some interpreters are able to translate a source code language into an intermediate language.



Let's Practice!

Laura talks about applications for interpreters.

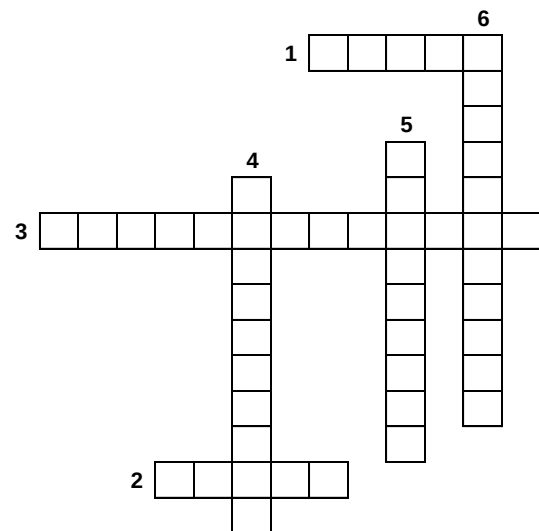
Instructions: Read the clues. Complete the crossword.

Across

1. In some cases, interpreters _____ the code and they can set its behavior instantly.
2. There is a _____ of interpreters I know.
3. Interpreters can run command languages or _____.

Down

4. Interpreters apply relevant security standards with _____ technology.
5. Some interpreters are able to _____ a source code language into an intermediate language.
6. An interpreter is a program that can run _____ without passing through a compilation process before.





Extension Activity

Let's design an interpreter.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Imagine you are a programmer.
- Choose a variation of the types of interpreters.
- Look for information about the one you chose.
- Prepare an oral presentation about it for your classmates.
- Present your job to your class.

Self-Assessment

Yes

No

- I choose a variation of the types of interpreters.
- I prepare an oral presentation for my classmates.
- I present my job to my classmates.

☐ ☐
☐ ☐
☐ ☐


Target Words

- **intermediate language** (adjective phrase): a representation of source code after an interpretation or compilation process.
e.g. Some interpreters are able to translate a source code language into an intermediate language.
- **parse** (verb): to make an analysis of the parts of an structure.
e.g. In some cases, they parse the code and they can set its behavior instantly.
- **bunch** (noun): refers to a group of objects or people.
e.g. Well, there is a bunch of interpreters I know about.
- **concern** (verb): to be related with something.
e.g. There are mainly four applications that concern them.
- **glue language** (adjective phrase): a type of scripting language.
e.g. First, interpreters can run command languages or glue languages.
- **sandboxing** (noun/verb): refers to a security process that separates a program from other in case of representing a risk for other programs.
e.g. Third, they allow virtualization and lastly they apply relevant security standards with sandboxing technology.



Learning Points

There is / There are in Affirmative and Interrogative Forms

In order to express the idea of the existence of something both in singular and plural forms, the English language combines the word **there** and the Verb To **Be**.

You use **there is** to refer to singular or uncountable nouns, and **there are** to refer to plural nouns:

- *There is* an instant interpretation of source code instructions.
- *There are* different kinds of interpreters.

The interrogative form is used to ask for **details in the information**:

- *Is there* an instant interpretation of source code instructions?
- *Are there* different kinds of interpreters?

When making questions, you use **any** when you want to inquire about objects in plural or when you are mentioning uncountable nouns:

- *Is there any* interpreter for precompiled code?
- *Are there any* variations of interpreters?

Affirmative form	There is - present singular form
	There is + noun (singular)+ complement
	There are - present plural form
	There are + noun (plural)+complement

Questions	Is there - present singular form
	Is there + a/an + countable noun (singular) + complement?
	Is there any+ uncountable noun (singular) + complement?
	Are there - present plural form
	Are there + noun (plural) +complement? Are there any + noun (plural) +complement?



Learning Points

Examples

- *Is there any translation process?*
- *Are there any specific applications referring to interpreters?*
- *Well, there is a bunch of interpreters I know. We have LISP and Darmouth Basic among those interpreters that do parsing and Phyton and Ruby can work for precompiled codes.*
- *There are some programs like Smalltalk that can create intermediate representations and execute precompiled data at the same time.*

Specific Information About Variations of Types of Interpreters

Interpreters can run source code without a previous compilation process.

Depending on their functional features, there are different types of interpreters:

Variations of Types of Interpreters	
Bytecode Interpreters	These interpreters translate source code into an intermediate language called bytecode. Bytecodes can be run in different computer systems and they are processed in virtual machines. One drawback of these interpreters is that they can't keep a global connection between the statements. Emacs Lisp is an example of bytecode.
Threaded Code interpreters	They generate pointers instead of bytecodes. Each word of a pointer contains an instruction connected to a specific task or sequences of tasks. A threaded code interpreter processes instruction by instruction. An example of this type of code is Forth.
Abstract Syntax Tree interpreters	AST are faster than other interpreters since they parse each statement just once and they are able to keep the global structure of a source code. However, AST can sometimes lead to overloading.
Just-in-time compilation	JIT compilation allows transforming an intermediate representation of a code to a computer language at the same time of a program's execution. Some examples are Javascript and NET framework.
Self-interpreter	This kind of program possesses a computer language that can interpret itself. This means that the interpreter needs to implement the same language in its system as the expected in the output code. Basic can be an example of this type of interpreter.

